

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH



THIẾT KẾ VI MẠCH (CO3098)

Bài Thực Hành 2

DIGITAL DESIGN SYNTHESIS USING CADENCE GENUS

Lớp: L04 - Nhóm: 08 - Học Kỳ HK252
Ngày nộp: 12/04/2026

GVHD: Thầy Phạm Kiều Nhật Anh

Sinh Viên	MSSV
Phạm Công Võ	2313946
Phạm Minh Đức	2311079
Nguyễn Trần Như Phước	2212719
Nguyễn Thanh Tùng	2112711

TP.Hồ Chí Minh, Tháng 04/2026

Mục lục

1	Giới thiệu	2
1.1	Mục tiêu Lab	2
1.2	Công cụ sử dụng	2
2	Môi trường thực hiện	3
2.1	Thông tin sinh viên và đường dẫn làm việc	3
2.2	Cấu trúc thư mục	3
2.3	File Constraint: <code>bound_flasher_gate.sdc</code>	4
3	LAB1 – Basic Genus Flow	4
3.1	Quy trình thực hiện	4
3.2	Netlist Schematic	6
3.3	Kết quả QoR - LAB1	6
4	LAB3 – Low-Power Synthesis	7
4.1	Cấu hình Low-Power	7
4.2	Kết quả và so sánh với LAB1	8
5	Tìm tần số hoạt động tối đa ($f_{\max}$)	9
5.1	Phương pháp	9
5.2	Kết quả thực nghiệm	9
5.3	File kết quả nộp bài tại $f_{\max}$	10

1 Giới thiệu

1.1 Mục tiêu Lab

Bài thực hành số 2 tập trung vào quy trình **tổng hợp logic** (*Logic Synthesis*) - giai đoạn quan trọng trong chu trình thiết kế vi mạch số, chuyển đổi mô tả RTL (*Register Transfer Level*) thành netlist cấp cổng (*Gate-Level Netlist*) sử dụng công cụ **Cadence Genus Synthesis Solution**.

Quy trình synthesis trong bài thực hành được chia thành ba phần:

1. **LAB1 – Basic Genus Flow:** Thực hiện synthesis cơ bản, làm quen với luồng công cụ và phân tích kết quả.
2. **LAB3 – Low-Power Synthesis:** Áp dụng kỹ thuật tối ưu điện năng và so sánh với LAB1.
3. **Tìm $f_{\max}$:** Xác định tần số hoạt động tối đa của thiết kế thông qua điều chỉnh timing constraint.

Sau khi hoàn thành bài thực hành, sinh viên có khả năng:

- ▶ Xây dựng môi trường synthesis chuẩn công nghiệp, bao gồm cấu trúc thư mục, thiết lập license và thực thi synthesis từ command-line với lệnh `genus -f run.tcl`.
- ▶ Viết và hiệu chỉnh **file constraint SDC** (*Synopsys Design Constraints*) để định nghĩa clock period, input/output delay và các điều kiện timing của thiết kế.
- ▶ Phân tích ba loại synthesis report: `final_area.rpt`, `final_qor.rpt` và `final_time.rpt`.
- ▶ Áp dụng Low-Power Synthesis thông qua Clock Gating và các thuộc tính tối ưu leakage/dynamic power, so sánh kết quả PPA (*Power - Performance - Area*) với synthesis thông thường.
- ▶ Xác định tần số hoạt động tối đa $f_{\max}$ bằng cách giảm dần clock period và kiểm tra Critical Path Slack ($CPS \geq 0$).

1.2 Công cụ sử dụng

Công cụ		Phiên bản	Vai trò
Cadence Genus		22.1	Tổng hợp logic RTL-to-Gate, thực thi qua script TCL (<code>genus -f run.tcl</code>), tối ưu diện tích, tốc độ và điện năng.
Genus GUI		22.1	Giao diện đồ họa xem <i>Netlist Schematic</i> và kiểm tra kết nối logic sau synthesis.
Cadence Xcelium		2021	Simulation RTL tại Lab 1, sinh ra file <code>bound_flasher.v</code> làm đầu vào cho quá trình synthesis.
Standard Cell Library	Cell	gsclib045	Thư viện cell chuẩn 45 nm, bao gồm file timing (<code>slow.lib</code> , <code>p11.lib</code>) và file vật lý LEF.

Bảng 1: Danh sách công cụ sử dụng trong Bài thực hành 2

Mối quan hệ giữa các công cụ trong quy trình được tóm tắt như sau: **Xcelium** (Lab 1) → RTL .v → **Genus** (synthesis) → Netlist .v → **Genus GUI** (kiểm tra schematic).

2 Môi trường thực hiện

2.1 Thông tin sinh viên và đường dẫn làm việc

Thông tin	Giá trị
Group	104group8
Student ID	2313946
Thư mục gốc	~/vlsi/2313946/work/
File RTL nguồn	simulation_env/bound_flasher.v
Thư mục LAB2	synthesis_env/Genus_BoundFlasher/

Bảng 2: Thông tin sinh viên và đường dẫn làm việc

2.2 Cấu trúc thư mục

Theo hướng dẫn của lab, trước khi bắt đầu copy toàn bộ *Sample Environment Kit* từ Cadence về thư mục làm việc:

```
1 cd ~/vlsi/2313946/work/synthesis_env
2 cp -rf /home/share_file/cadence/installs/Genus_CUI_RAK/ ./Genus_BoundFlasher
```

Sau khi copy, cấu trúc thư mục làm việc như sau:

```
1 ~/vlsi/2313946/work/
2 |-- simulation_env/
3 |   |-- bound_flasher.v      <- File RTL nguồn (tu Lab 1)
4 |   '-- bound_flasher_tb.v
5 '-- synthesis_env/
6   '-- Genus_BoundFlasher/    <- Thư mục làm việc chính
7       |-- RTL/
8       |   '-- bound_flasher.v  <- Copy RTL vào đây
9       |-- LIB/                 <- Thư viện cell (*.lib)
10      |-- LEF/                 <- File LEF vật lý
11      |-- constraints/
12      |   '-- bound_flasher_gate.sdc
13      |-- LAB1/
14      |   |-- run.tcl
15      |   |-- gui.tcl
16      |   |-- sync.log
17      |   |-- outputs_{DATE}/
18      |   |   '-- bound_flasher_m.v  <- Netlist output
19      |   '-- reports_{DATE}/
20      |       |-- final_area.rpt
21      |       |-- final_qor.rpt
22      |       '-- final_time.rpt
23      '-- LAB3/
24          |-- run.tcl
25          '-- reports_{DATE}/
```

Tiếp theo, copy file RTL từ `simulation_env` sang thư mục RTL/:

```
1 cp ../simulation_env/bound_flasher.v ./Genus_BoundFlasher/RTL/
```

2.3 File Constraint: bound_flasher_gate.sdc

Theo hướng dẫn lab, file SDC được tạo tại đường dẫn `constraints/bound_flasher_gate.sdc` để định nghĩa clock và timing cho quá trình synthesis. Nội dung file như sau:

```
1 # Set the current design
2 current_design bound_flasher
3
4 create_clock -name "clk" -add -period 5.0 \
5             -waveform {0.0 2.5} [get_ports clk]
6
7 set_input_delay -clock [get_clocks clk] \
8               -add_delay 2.5 [get_ports flick]
9 set_input_delay -clock [get_clocks clk] \
10              -add_delay 2.5 [get_ports rst_n]
11 set_output_delay -clock [get_clocks clk] \
12                -add_delay 2.5 [get_ports lamp]
13
14 set_max_fanout 15.000 [current_design]
15 set_max_transition 1.2 [current_design]
```

Bảng 3 giải thích các tham số quan trọng trong file SDC:

Tham số	Ý nghĩa
-period 5.0	Chu kỳ clock = 5 ns $\Rightarrow$ tần số khởi đầu 200 MHz
-waveform {0.0 2.5}	Cạnh lên tại 0 ns, cạnh xuống tại 2.5 ns (duty cycle 50%)
set_input_delay 2.5	Delay đầu vào = 50% chu kỳ clock
set_output_delay 2.5	Delay đầu ra = 50% chu kỳ clock
set_max_fanout 15	Giới hạn fanout tối đa của mỗi net trong thiết kế
set_max_transition 1.2	Giới hạn thời gian chuyển trạng thái tối đa (ns)

Bảng 3: Giải thích các tham số trong file SDC

Lưu ý quan trọng: Khi thay đổi clock period để tìm $f_{\max}$, giá trị `waveform` và `delay` phải luôn bằng **50%** của period để đảm bảo duty cycle 50%. Ví dụ: period = 3.0 ns $\Rightarrow$ waveform {0.0 1.5}, delay = 1.5 ns.

3 LAB1 – Basic Genus Flow

3.1 Quy trình thực hiện

1. **Sửa file run.tcl:** File `run.tcl` có sẵn trong kit nhưng cấu hình cho design mẫu, cần chỉnh sửa lại 4 tham số cho đúng với `bound_flasher`:

```
1 # Chỉnh sửa 4 cho sau trong run.tcl:
2 set DESIGN bound_flasher           ; # Ten module top-level
3 read_hdl "bound_flasher.v"         ; # Ten file RTL
```

```
4 read_sdc ../constraints/bound_flasher_gate.sdc ; # File SDC
5 write_hdl > ${_OUTPUTS_PATH}/${_DESIGN}_m.v ; # Output netlist
```

2. Cấp license và chạy Synthesis: Thực hiện lần lượt các lệnh sau:

```
1 cd /home/share_file/cadence
2 source add_path
3 source add_license
4 cd ~/vlsi/2313946/work/synthesis_env/Genus_BoundFlasher/LAB1
5 genus -f run.tcl | tee -i sync.log
```

Lệnh `tee -i sync.log` vừa hiển thị output ra terminal vừa lưu vào file log. Quá trình synthesis mất khoảng 1-3 phút và kết thúc với thông báo **Synthesis Finished** và **Normal exit**.

3. Kiểm tra Log file: Mở sync.log và tìm từ khóa Error để xác nhận không có lỗi:

```
1 vi sync.log
2 /Error
```

Nếu terminal báo **Pattern not found** thì synthesis thành công, tiếp tục kiểm tra report.

4. Kiểm tra Synthesis Report: Ba loại report được sinh ra trong thư mục `reports_{DATE}/`:

```
1 vi reports*/final_area.rpt # Kiểm tra diện tích
2 vi reports*/final_qor.rpt # Kiểm tra QoR
3 vi reports*/final_time.rpt # Kiểm tra timing (quan trọng nhất)
```

5. Xem Netlist Schematic (GUI): Tạo file gui.tcl với nội dung sau:

```
1 set DESIGN bound_flasher
2
3 read_libs "../LIB/slow.lib ../LIB/p11.lib \
4           ../LIB/CDK_S128x16.lib \
5           ../LIB/CDK_S256x16.lib \
6           ../LIB/CDK_R512x16.lib"
7
8 read_physical -lef "../LEF/gsclib045_tech.lef \
9                ../LEF/gsclib045_macro.lef \
10               ../LEF/p11.lef \
11               ../LEF/CDK_S128x16.lef \
12               ../LEF/CDK_S256x16.lef \
13               ../LEF/CDK_R512x16.lef"
14
15 read_hdl "../outputs_Apr10-XX:XX:XX/bound_flasher_m.v"
16
17 elaborate $DESIGN
```

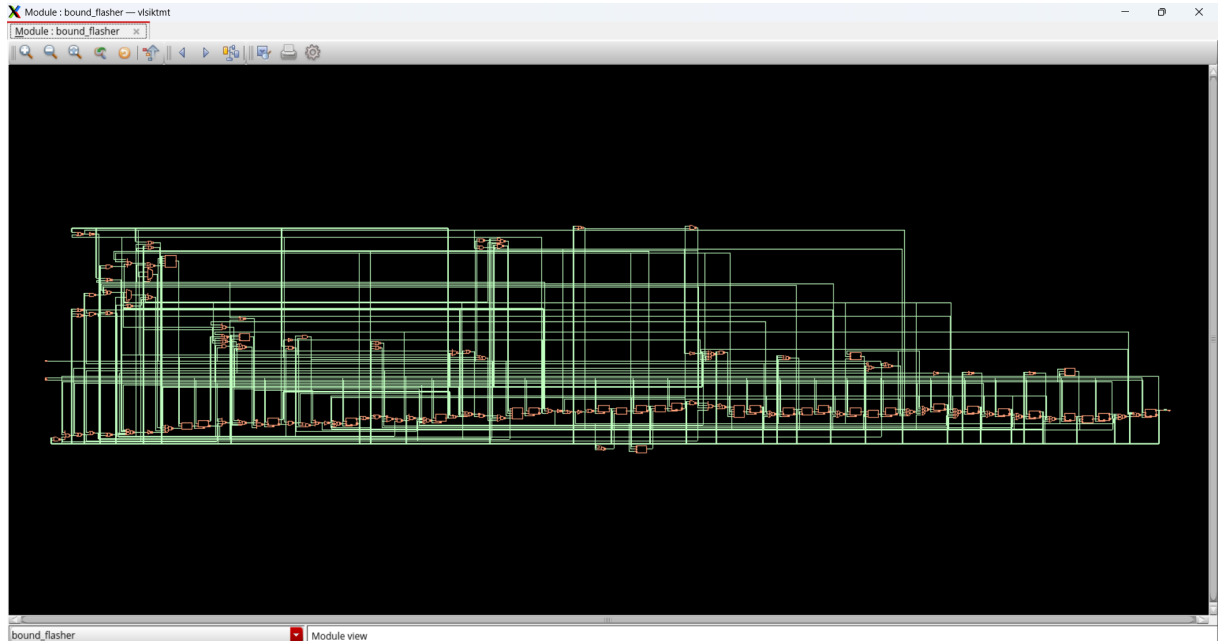
Sau đó mở Genus GUI bằng lệnh:

```
1 genus -f gui.tcl -gui
```

Trong cửa sổ GUI: **Right Click** vào module `bound_flasher` → chọn **Schematic** để xem sơ đồ netlist.

3.2 Netlist Schematic

Netlist được nạp vào Genus GUI qua lệnh `genus -f gui.tcl -gui`, sau đó **Right Click** → **Schematic** để xem sơ đồ kết nối logic gồm **125 LeafCells** và **144 Nets**.



Hình 1: Netlist Schematic của *bound_flasher* – LAB1 Basic Synthesis

3.3 Kết quả QoR - LAB1

Sau khi synthesis hoàn thành, kiểm tra `final_qor.rpt` xác nhận thông tin timing tổng quan của thiết kế tại chu kỳ clock 5000 ps (5.0 ns):

```
Timing
-----

Clock Period
-----
clk  5000.0


```

Cost Group	Critical Path Slack	TNS	Violating Paths
clk	1556.3	0.0	0
default	No paths	0.0	
Total		0.0	0

Hình 2: QoR Report – LAB1 Basic Synthesis (Clock Period = 5000 ps, CPS = 1556.3 ps)

Kết quả cho thấy:

- ▶ Clock clk với chu kỳ **5000.0 ps** ($5.0 \text{ ns} \Rightarrow 200 \text{ MHz}$).
- ▶ Critical Path Slack = **1556.3 ps** $\geq 0 \Rightarrow$ **PASS**, netlist hợp lệ, đạt yêu cầu timing.
- ▶ TNS (*Total Negative Slack*) = **0.0 ps** và Violating Paths = **0** $\Rightarrow$ không có vi phạm timing nào.

4 LAB3 – Low-Power Synthesis

4.1 Cấu hình Low-Power

LAB3 thực hiện tương tự LAB1 nhưng bổ sung các thuộc tính tối ưu điện năng vào `run.tcl`. Các thuộc tính được thêm vào theo thứ tự:

Trước lệnh `elaborate`:

```
1 set_db / .lp_insert_clock_gating true
2 set_db / .leakage_power_effort medium
```

Sau lệnh `elaborate`:

```
1 set_db "design:$DESIGN" .max_leakage_power 0.0
2 set_db "design:$DESIGN" .lp_power_optimization_weight 0.5
3 set_db "design:$DESIGN" .max_dynamic_power 100
```

Thuộc tính	Giải thích
lp_insert_clock_gating true	Bật Clock Gating – tắt clock khi module không hoạt động để tiết kiệm dynamic power. Phải đặt trước lệnh <code>elaborate</code> .
leakage_power_effort medium	Mức độ nỗ lực tối ưu leakage power: low / medium / high.
max_leakage_power 0.0	Đặt mục tiêu leakage power = 0 (tối ưu tối đa có thể).
lp_power_optimization_weight 0.5	Trọng số cân bằng: 0 = ưu tiên dynamic power, 1 = ưu tiên leakage power. Giá trị 0.5 = cân bằng 50/50.
max_dynamic_power 100	Giới hạn dynamic power tối đa = $100 \mu\text{W}$.

Bảng 4: Giải thích các thuộc tính Low-Power trong LAB3

Sau khi sửa `run.tcl`, chạy synthesis LAB3:

```
1 cd /home/share_file/cadence
2 source add_path
3 source add_license
4 cd ~/vlsi/2313946/work/synthesis_env/Genus_BoundFlasher/LAB3
5 genus -f run.tcl | tee -i sync.log
```

4.2 Kết quả và so sánh với LAB1

Thông số	LAB1 – Basic	LAB3 – Low-Power
Clock Period (ns)	5.0	5.0
Critical Path Slack (ps)	1556.3	408.9
Clock Gating	Không	Có
Kết quả Timing	PASS	PASS

Bảng 5: So sánh kết quả LAB1 (Basic) và LAB3 (Low-Power)

Nhận xét:

LAB3 có CPS = **408.9 ps** thấp hơn LAB1 (1556.3 ps) do Genus ưu tiên dùng cell tiêu thụ điện thấp hơn (thường chậm hơn). Sự xuất hiện của `cg_enable_group_clk` xác nhận Clock Gating đã được bật thành công. Cả hai đều đạt yêu cầu timing (Slack ≥ 0).

```
Timing
-----

Clock Period
-----
clk  5000.0
```

Cost Group	Critical Path Slack	TNS	Violating Paths
C2C	408.9	0.0	0
C20	1967.4	0.0	0
cg_enable_group_clk	1280.8	0.0	0
clk	No paths	0.0	
default	No paths	0.0	
I2C	239.4	0.0	0
I20	No paths	0.0	
Total		0.0	0

Hình 3: QoR Report - LAB3 Low-Power Synthesis (Clock Period = 5000 ps, CPS = 408.9 ps)

5 Tìm tần số hoạt động tối đa ($f_{\max}$)

5.1 Phương pháp

Tần số hoạt động tối đa được xác định bằng cách giảm dần clock period trong file SDC, chạy lại synthesis và kiểm tra **Critical Path Slack (CPS)**:

$$f_{\max} = \frac{1}{T_{\min}} \quad \text{với } T_{\min} \text{ là period nhỏ nhất mà CPS} \geq 0$$

Quy tắc điều chỉnh SDC: với mỗi giá trị period T , các tham số waveform và delay luôn bằng $T/2$ (duty cycle 50%).

5.2 Kết quả thực nghiệm

Lần	Period (ns)	Tần số	CPS (ps)	Kết quả
1	5.0	200 MHz	1556.3	PASS
2	3.5	286 MHz	668.0	PASS
3	3.0	333 MHz	168.0	PASS
4	2.5	400 MHz	16.0	PASS
5	1.5	667 MHz	1.0	PASS
6	1.37	729.9 MHz	0.0	PASS ($f_{\max}$)

Bảng 6: Kết quả thực nghiệm tìm $f_{\max}$ – giảm dần clock period

Timing			

Clock Period			

clk	1370.0		
Cost	Critical		Violating
Group	Path Slack	TNS	Paths

clk	0.0	0.0	0
default	No paths	0.0	

Total		0.0	0

Hình 4: QoR Report - Kết quả thực nghiệm tìm $f_{\max}$ (Clock Period = 1370.0ps, CPS = 0.0ps)

5.3 File kết quả nộp bài tại $f_{\max}$

Sau khi re-synthesize với $f_{\max} = 729.9 \text{ MHz}$ (period = 1.37 ns), bốn file kết quả được nộp gồm:

File	Đường dẫn
Netlist	outputs*/bound_flasher_m.v
Area Report	reports*/final_area.rpt
QoR Report	reports*/final_qor.rpt
Timing Report	reports*/final_time.rpt

Bảng 7: Bốn file kết quả nộp bài tại $f_{\max} = 729.9 \text{ MHz}$